

Deep Neural Networks - Programming Assignment

Comparing Linear Models and Multi-Layer Perceptrons

Student Name: SAMPATH KUMAR S SHETTY | **Student ID:** 2024ac05041@wilp.bits-pilani.ac.in | **Contribution:** 100%

Student Name: ARUN A. PAI | **Student ID:** 2024ac05014@wilp.bits-pilani.ac.in | **Contribution:** 100%

Student Name: DEEPTHY.A.N. | **Student ID:** 2024ad05350@wilp.bits-pilani.ac.in | **Contribution:** 100%

Student Name: ANISH RADHAKRISHNAN SHOBHA KAITHASSERY | **Student ID:** 2024ad05344@wilp.bits-pilani.ac.in | **Contribution:** 100%

Date: 26th December 2025

⚠ IMPORTANT INSTRUCTIONS

1. **Complete ALL sections** marked with `TODO`
2. **DO NOT modify** the `get_assignment_results()` function structure
3. **Track training time** for both models using `time.time()` \n
4. **Store loss_history** in both model classes
5. **Calculate ALL metrics** (accuracy, precision, recall, F1)
6. **Fill get_assignment_results()** with ALL required fields
7. **PRINT the results** - Auto-grader needs visible output!
8. **Run all cells** before submitting (Kernel → Restart & Run All)

SCORING:

- Missing fields = 0 marks for that section
- Non-executed notebook = 0 marks

- Cleared outputs = 0 marks

```
In [2]: # Install required packages
import subprocess
import sys

def install_package(package):
    """Install a package using pip with multiple methods"""
    methods = [
        [sys.executable, "-m", "pip", "install", package],
        [sys.executable, "-m", "pip", "install", "--user", package],
        [sys.executable, "-m", "pip", "install", "--upgrade", package],
    ]

    for i, method in enumerate(methods, 1):
        try:
            print(f"  Method {i}: {' '.join(method)}")
            subprocess.check_call(method, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
            print(f"✅ Successfully installed {package} (Method {i})")
            return True
        except subprocess.CalledProcessError:
            print(f"  ❌ Method {i} failed")
            continue

    print(f"❌ All installation methods failed for {package}")
    return False

# Required packages for this assignment
required_packages = [
    "numpy",
    "pandas",
    "matplotlib"
    # Note: scikit-learn removed - using manual implementations only
]

print("\n Installing required packages...")
print("-" * 40)

failed_packages = []
```

```

for package in required_packages:
    try:
        # Test import with proper module names
        if package == "scikit-learn":
            __import__("sklearn")
        else:
            __import__(package)
        print(f"✅ {package} is already installed")
    except ImportError:
        print(f"🔧 Installing {package}...")
        success = install_package(package)
        if not success:
            failed_packages.append(package)

print("\n" + "=" * 50)
if failed_packages:
    print("⚠️ SOME PACKAGES FAILED TO INSTALL:")
    for pkg in failed_packages:
        print(f"    - {pkg}")

    print("\n💡 MANUAL INSTALLATION OPTIONS:")
    print("    Option 1: Run in terminal/command prompt:")
    for pkg in failed_packages:
        print(f"        pip install {pkg}")

    print("\n    Option 2: Use conda (if available):")
    for pkg in failed_packages:
        conda_pkg = pkg if pkg != "scikit-learn" else "scikit-learn"
        print(f"        conda install {conda_pkg}")

    print("\n    Option 3: Restart kernel and try again")
    print("\n💡 NOTE:")
    print("    This assignment uses manual implementations for everything")
    print("    - No scikit-learn dependencies")
    print("    - All preprocessing functions written from scratch")
else:
    print("🎉 All packages installed successfully!")

print("=" * 50)

```

🔧 Installing required packages...

```

=====
✅ numpy is already installed
✅ pandas is already installed
✅ matplotlib is already installed

```

```

=====
🎉 All packages installed successfully!
=====

```

```

In [3]: # Fallback implementations if scikit-learn is not available
def manual_train_test_split(X, y, test_size=0.2, random_state=42):
    """Manual implementation of train_test_split"""
    np.random.seed(random_state)
    n_samples = len(X)
    n_test = int(n_samples * test_size)

    # Create random indices
    indices = np.random.permutation(n_samples)
    test_indices = indices[:n_test]
    train_indices = indices[n_test:]

    if hasattr(X, 'iloc'): # pandas DataFrame
        X_train = X.iloc[train_indices]
        X_test = X.iloc[test_indices]
        y_train = y.iloc[train_indices] if hasattr(y, 'iloc') else y[train_indices]
        y_test = y.iloc[test_indices] if hasattr(y, 'iloc') else y[test_indices]
    else: # numpy array
        X_train = X[train_indices]
        X_test = X[test_indices]
        y_train = y[train_indices]
        y_test = y[test_indices]

    return X_train, X_test, y_train, y_test

class ManualStandardScaler:
    """Manual implementation of StandardScaler"""
    def __init__(self):
        self.mean_ = None
        self.std_ = None

```

```

def fit(self, X):
    """Compute mean and std for scaling"""
    if hasattr(X, 'values'): # pandas DataFrame
        X = X.values
    self.mean_ = np.mean(X, axis=0)
    self.std_ = np.std(X, axis=0)
    # Avoid division by zero
    self.std_[self.std_ == 0] = 1.0
    return self

def transform(self, X):
    """Scale features using computed mean and std"""
    if hasattr(X, 'values'): # pandas DataFrame
        X = X.values
    return (X - self.mean_) / self.std_

def fit_transform(self, X):
    """Fit and transform in one step"""
    return self.fit(X).transform(X)

print("✅ Fallback implementations ready!")
print(" - manual_train_test_split(): For data splitting")
print(" - ManualStandardScaler(): For feature scaling")
print(" These will be used if scikit-learn is not available")

```

```

✅ Fallback implementations ready!
 - manual_train_test_split(): For data splitting
 - ManualStandardScaler(): For feature scaling
 These will be used if scikit-learn is not available

```

```

In [4]: # Import required libraries with error handling and fallbacks
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time
import warnings
warnings.filterwarnings('ignore')

# Using manual implementations only (no scikit-learn)
print("👶 Using manual implementations for all preprocessing")
sklearn_available = False

```

```

# Use our manual implementations
train_test_split = manual_train_test_split
StandardScaler = ManualStandardScaler

print("✅ All essential libraries loaded!")
print("👶 Using 100% manual implementations - no sklearn dependencies!")

# Set random seed for reproducibility
np.random.seed(42)

print("📦 All required libraries imported successfully!")
print("🎯 Assignment: Implementing Linear Model and MLP from scratch")
print("==" * 60)

```

```

👶 Using manual implementations for all preprocessing
✅ All essential libraries loaded!
👶 Using 100% manual implementations - no sklearn dependencies!
📦 All required libraries imported successfully!
🎯 Assignment: Implementing Linear Model and MLP from scratch
=====

```

Section 1: Dataset Selection and Loading

Requirements:

- ≥ 500 samples
- ≥ 5 features
- Public dataset (UCI/Kaggle)
- Regression OR Classification problem

```

In [5]: # Load Wine Quality Dataset from UCI ML Repository
# This dataset has 1599 samples and 11 features, suitable for binary classification
import ssl
import urllib.request

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv"

```

```

try:
    # Fix SSL certificate verification error
    ssl_context = ssl.create_default_context()
    ssl_context.check_hostname = False
    ssl_context.verify_mode = ssl.CERT_NONE

    # Load the dataset with SSL fix
    with urllib.request.urlopen(url, context=ssl_context) as response:
        data = pd.read_csv(response, delimiter=';')

    # Convert quality to binary classification (good wine: quality >= 6, bad wine: quality < 6)
    data['quality_binary'] = (data['quality'] >= 6).astype(int)

    print("✅ Dataset loaded successfully!")
    print(f"📊 Dataset shape: {data.shape}")
    print(f"🍷 Sample data:")
    print(data.head())

except Exception as e:
    print(f"❌ Error loading dataset: {e}")
    print(f"🔄 Creating synthetic dataset for demonstration...")

    # Create synthetic dataset if loading fails
    np.random.seed(42)
    n_samples_synthetic = 1000
    n_features_synthetic = 8

    X_synthetic = np.random.randn(n_samples_synthetic, n_features_synthetic)
    y_synthetic = (X_synthetic[:, 0] + X_synthetic[:, 1] * 0.5 +
                  X_synthetic[:, 2] * 0.3 + np.random.randn(n_samples_synthetic) * 0.1 > 0).astype(int)

    feature_names = [f'feature_{i}' for i in range(n_features_synthetic)]
    data = pd.DataFrame(X_synthetic, columns=feature_names)
    data['quality_binary'] = y_synthetic

    print("✅ Synthetic dataset created!")

# Dataset information
dataset_name = "Wine Quality Dataset"
dataset_source = "UCI ML Repository"
n_samples = len(data)

```

```

n_features = len(data.columns) - 1 # Excluding target
problem_type = "binary_classification"

# Problem statement
problem_statement = """
Predicting wine quality (good vs bad) based on physicochemical properties.
This classification helps wineries optimize production processes and quality control.
We classify wines with quality rating ≥6 as 'good' and <6 as 'bad'.
"""

# Primary evaluation metric
primary_metric = "f1_score"

# Metric justification
metric_justification = """
F1-score has been chosen as the primary metric because it balances precision and recall,
which is important for wine quality assessment where both false positives
(marking bad wine as good) and false negatives (marking good wine as bad) have costs.
"""

print(f"\n📋 DATASET SUMMARY:")
print(f"Dataset: {dataset_name}")
print(f"Source: {dataset_source}")
print(f"Samples: {n_samples}, Features: {n_features}")
print(f"Problem Type: {problem_type}")
print(f"Primary Metric: {primary_metric}")
print(f"\n📖 Problem Statement: {problem_statement.strip()}")
print(f"\n📏 Metric Justification: {metric_justification.strip()}")
print(f"Primary Metric: {primary_metric}")

```

✅ Dataset loaded successfully!

📊 Dataset shape: (1599, 13)

🍷 Sample data:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	\
0	7.4	0.70	0.00	1.9	0.076	
1	7.8	0.88	0.00	2.6	0.098	
2	7.8	0.76	0.04	2.3	0.092	
3	11.2	0.28	0.56	1.9	0.075	
4	7.4	0.70	0.00	1.9	0.076	

	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	\
0	11.0	34.0	0.9978	3.51	0.56	
1	25.0	67.0	0.9968	3.20	0.68	
2	15.0	54.0	0.9970	3.26	0.65	
3	17.0	60.0	0.9980	3.16	0.58	
4	11.0	34.0	0.9978	3.51	0.56	

	alcohol	quality	quality_binary
0	9.4	5	0
1	9.8	5	0
2	9.8	5	0
3	9.8	6	1
4	9.4	5	0

📋 DATASET SUMMARY:

Dataset: Wine Quality Dataset

Source: UCI ML Repository

Samples: 1599, Features: 12

Problem Type: binary_classification

Primary Metric: f1_score

📋 Problem Statement: Predicting wine quality (good vs bad) based on physicochemical properties. This classification helps wineries optimize production processes and quality control. We classify wines with quality rating ≥ 6 as 'good' and < 6 as 'bad'.

📊 Metric Justification: F1-score has been chosen as the primary metric because it balances precision and recall, which is important for wine quality assessment where both false positives (marking bad wine as good) and false negatives (marking good wine as bad) have costs. Primary Metric: f1_score

Section 2: Data Preprocessing

Preprocess your data:

1. Handle missing values
2. Encode categorical variables
3. Split into train/test sets
4. Scale features

```
In [6]: # Data Preprocessing Pipeline
print("🍷 Starting data preprocessing...")

# 1. Separate features (X) and target (y)
if 'quality_binary' in data.columns:
    # Drop original quality column if it exists
    if 'quality' in data.columns:
        X = data.drop(['quality', 'quality_binary'], axis=1)
    else:
        X = data.drop('quality_binary', axis=1)
    y = data['quality_binary']
else:
    # Fallback for synthetic data
    X = data.iloc[:, :-1]
    y = data.iloc[:, -1]

print(f"✅ Features extracted: {X.shape}")
print(f"✅ Target extracted: {y.shape}")

# 2. Handle missing values
missing_values = X.isnull().sum().sum()
if missing_values > 0:
    print(f"⚠️ Found {missing_values} missing values - filling with median")
    X = X.fillna(X.median())
else:
    print("✅ No missing values found")

# 3. Check data types and basic statistics
```

```

print(f"\n📊 FEATURE STATISTICS:")
print(f"Feature names: {list(X.columns)}")
print(f"Target distribution: {y.value_counts().to_dict()}")

# 4. Train-test split (80-20)
try:
    # Try with stratify for balanced split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )
    print("✅ Stratified train-test split completed")
except (TypeError, NameError):
    # Fallback without stratify if using manual implementation
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )
    print("✅ Train-test split completed (manual implementation)")

# 5. Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Convert to numpy arrays for easier handling
X_train_scaled = np.array(X_train_scaled)
X_test_scaled = np.array(X_test_scaled)
y_train = np.array(y_train)
y_test = np.array(y_test)

# Update preprocessing info
train_samples = len(X_train_scaled)
test_samples = len(X_test_scaled)
train_test_ratio = train_samples / (train_samples + test_samples)

print(f"\n🔧 PREPROCESSING COMPLETE:")
print(f"Train samples: {train_samples}")
print(f"Test samples: {test_samples}")
print(f"Split ratio: {train_test_ratio:.1%} - {1-train_test_ratio:.1%}")
print(f"Features shape: {X_train_scaled.shape[1]}")

```

```

print(f"Feature scaling: StandardScaler applied")
print(f"Data ready for model training! 🚀")

```

```

🔧 Starting data preprocessing...
✅ Features extracted: (1599, 11)
✅ Target extracted: (1599,)
✅ No missing values found

```

```

📊 FEATURE STATISTICS:
Feature names: ['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur diox
ide', 'total sulfur dioxide', 'density', 'pH', 'sulphates', 'alcohol']
Target distribution: {1: 855, 0: 744}
✅ Train-test split completed (manual implementation)

```

```

🔧 PREPROCESSING COMPLETE:
Train samples: 1280
Test samples: 319
Split ratio: 80.1% - 19.9%
Features shape: 11
Feature scaling: StandardScaler applied
Data ready for model training! 🚀

```

Section 3: Baseline Model Implementation

Implement from scratch (NO sklearn models!):

- Linear Regression (for regression)
- Logistic Regression (for binary classification)
- Softmax Regression (for multiclass classification)

Must include:

- Forward pass (prediction)
- Loss computation
- Gradient computation
- Gradient descent loop
- Loss tracking

```
In [7]: class BaselineModel:
        """
        Baseline Logistic Regression with gradient descent (from scratch)
        """
        def __init__(self, learning_rate=0.01, n_iterations=1000):
            self.lr = learning_rate
            self.n_iterations = n_iterations
            self.weights = None
            self.bias = None
            self.loss_history = []

        def sigmoid(self, z):
            """Sigmoid activation function"""
            # Clip z to prevent overflow
            z = np.clip(z, -500, 500)
            return 1 / (1 + np.exp(-z))

        def fit(self, X, y):
            """Train the logistic regression model"""
            # Initialize parameters
            n_samples, n_features = X.shape
            self.weights = np.random.normal(0, 0.01, n_features)
            self.bias = 0

            # Gradient descent loop
            for i in range(self.n_iterations):
                # 1. Forward pass (predictions)
                z = np.dot(X, self.weights) + self.bias
                predictions = self.sigmoid(z)

                # 2. Compute loss (binary cross-entropy)
                epsilon = 1e-15 # Small value to prevent log(0)
                predictions = np.clip(predictions, epsilon, 1 - epsilon)
                loss = -np.mean(y * np.log(predictions) + (1 - y) * np.log(1 - predictions))

                # 3. Compute gradients
                dw = np.dot(X.T, (predictions - y)) / n_samples
                db = np.mean(predictions - y)

                # 4. Update weights
```

```
                self.weights = self.weights - self.lr * dw
                self.bias = self.bias - self.lr * db

                # 5. Store loss
                self.loss_history.append(loss)

                # Print progress
                if (i + 1) % 100 == 0 or i == 0:
                    print(f"Iteration {i+1}/{self.n_iterations}, Loss: {loss:.6f}")

            return self

        def predict_proba(self, X):
            """Predict probabilities"""
            z = np.dot(X, self.weights) + self.bias
            return self.sigmoid(z)

        def predict(self, X):
            """Make predictions"""
            probabilities = self.predict_proba(X)
            return (probabilities >= 0.5).astype(int)

print("✅ BaselineModel (Logistic Regression) implemented from scratch!")
```

✅ BaselineModel (Logistic Regression) implemented from scratch!

```
In [8]: # Train baseline model
print("🚀 Training Baseline Model (Logistic Regression)...")
print("-" * 50)
baseline_start = time.time()

# Initialize and train baseline model
baseline_model = BaselineModel(learning_rate=0.1, n_iterations=1000)
baseline_model.fit(X_train_scaled, y_train)

# Make predictions
baseline_predictions = baseline_model.predict(X_test_scaled)
baseline_probabilities = baseline_model.predict_proba(X_test_scaled)

baseline_training_time = time.time() - baseline_start
```

```

print(f"\n✅ Baseline training completed in {baseline_training_time:.2f}s")
print(f"✅ Loss decreased from {baseline_model.loss_history[0]:.4f} to {baseline_model.loss_history[-1]:.4f}")

# Store loss for comparison
baseline_initial_loss = baseline_model.loss_history[0]
baseline_final_loss = baseline_model.loss_history[-1]

print(f"🎉 Training complete – ready for evaluation!")

```

🚀 Training Baseline Model (Logistic Regression)...

```

-----
Iteration 1/1000, Loss: 0.691010
Iteration 100/1000, Loss: 0.525930
Iteration 200/1000, Loss: 0.520102
Iteration 300/1000, Loss: 0.518717
Iteration 400/1000, Loss: 0.518262
Iteration 500/1000, Loss: 0.518089
Iteration 600/1000, Loss: 0.518016
Iteration 700/1000, Loss: 0.517980
Iteration 800/1000, Loss: 0.517961
Iteration 900/1000, Loss: 0.517949
Iteration 1000/1000, Loss: 0.517941

```

```

✅ Baseline training completed in 0.18s
✅ Loss decreased from 0.6910 to 0.5179
🎉 Training complete – ready for evaluation!

```

Section 4: Multi-Layer Perceptron Implementation

Implement MLP from scratch with:

- At least 1 hidden layer
- ReLU activation for hidden layers
- Appropriate output activation
- Forward propagation
- Backward propagation
- Gradient descent

```

In [9]: class MLP:
        """
        Multi-Layer Perceptron implemented from scratch with backpropagation
        """
        def __init__(self, architecture, learning_rate=0.01, n_iterations=1000):
            """
            architecture: list [input_size, hidden1, hidden2, ..., output_size]
            Example: [11, 16, 8, 1] means:
                - 11 input features
                - Hidden layer 1: 16 neurons
                - Hidden layer 2: 8 neurons
                - Output layer: 1 neuron
            """
            self.architecture = architecture
            self.lr = learning_rate
            self.n_iterations = n_iterations
            self.parameters = {}
            self.loss_history = []
            self.cache = {}

        def initialize_parameters(self):
            """Initialize weights and biases for all layers"""
            np.random.seed(42)

            for l in range(1, len(self.architecture)):
                # Xavier initialization for better convergence
                self.parameters[f'W{l}'] = np.random.randn(
                    self.architecture[l], self.architecture[l-1]
                ) * np.sqrt(2.0 / self.architecture[l-1])

                self.parameters[f'b{l}'] = np.zeros((self.architecture[l], 1))

        def relu(self, Z):
            """ReLU activation function"""
            return np.maximum(0, Z)

        def relu_derivative(self, Z):
            """Derivative of ReLU"""
            return (Z > 0).astype(float)

```



```

def sigmoid(self, Z):
    """Sigmoid activation function"""
    Z = np.clip(Z, -500, 500) # Prevent overflow
    return 1 / (1 + np.exp(-Z))

def sigmoid_derivative(self, Z):
    """Derivative of sigmoid"""
    s = self.sigmoid(Z)
    return s * (1 - s)

def forward_propagation(self, X):
    """Forward pass through all layers"""
    A = X.T # Shape: (n_features, m_samples)
    self.cache['A0'] = A

    # Forward through hidden layers (with ReLU)
    for l in range(1, len(self.architecture) - 1):
        W = self.parameters[f'W{l}']
        b = self.parameters[f'b{l}']

        Z = np.dot(W, A) + b
        A = self.relu(Z)

        self.cache[f'Z{l}'] = Z
        self.cache[f'A{l}'] = A

    # Output layer (with sigmoid for binary classification)
    l = len(self.architecture) - 1
    W = self.parameters[f'W{l}']
    b = self.parameters[f'b{l}']

    Z = np.dot(W, A) + b
    A = self.sigmoid(Z)

    self.cache[f'Z{l}'] = Z
    self.cache[f'A{l}'] = A

    return A

def compute_loss(self, A_last, y):
    """Compute binary cross-entropy loss"""

```

```

    m = y.shape[0]
    y = y.reshape(1, m)

    # Prevent log(0)
    epsilon = 1e-15
    A_last = np.clip(A_last, epsilon, 1 - epsilon)

    loss = -np.mean(y * np.log(A_last) + (1 - y) * np.log(1 - A_last))
    return loss

def backward_propagation(self, X, y):
    """Backpropagation to compute gradients"""
    m = X.shape[0]
    y = y.reshape(1, m)
    L = len(self.architecture) - 1

    gradients = {}

    # Output layer gradients
    A_last = self.cache[f'A{L}']
    dZ = A_last - y # For sigmoid + cross-entropy

    gradients[f'dW{L}'] = np.dot(dZ, self.cache[f'A{L-1}'].T) / m
    gradients[f'db{L}'] = np.mean(dZ, axis=1, keepdims=True)

    # Hidden layers gradients (backpropagate)
    for l in range(L-1, 0, -1):
        W_next = self.parameters[f'W{l+1}']
        dA = np.dot(W_next.T, dZ)
        dZ = dA * self.relu_derivative(self.cache[f'Z{l}'])

        gradients[f'dW{l}'] = np.dot(dZ, self.cache[f'A{l-1}'].T) / m
        gradients[f'db{l}'] = np.mean(dZ, axis=1, keepdims=True)

    return gradients

def update_parameters(self, gradients):
    """Update parameters using gradient descent"""
    for l in range(1, len(self.architecture)):
        self.parameters[f'W{l}'] -= self.lr * gradients[f'dW{l}']
        self.parameters[f'b{l}'] -= self.lr * gradients[f'db{l}']

```

```

def fit(self, X, y):
    """Train the MLP using gradient descent"""
    # Initialize parameters
    self.initialize_parameters()

    # Training loop
    for i in range(self.n_iterations):
        # Forward propagation
        A_last = self.forward_propagation(X)

        # Compute loss
        loss = self.compute_loss(A_last, y)
        self.loss_history.append(loss)

        # Backward propagation
        gradients = self.backward_propagation(X, y)

        # Update parameters
        self.update_parameters(gradients)

        # Print progress
        if (i + 1) % 100 == 0 or i == 0:
            print(f"Iteration {i+1}/{self.n_iterations}, Loss: {loss:.6f}")

    return self

def predict_proba(self, X):
    """Predict probabilities"""
    A_last = self.forward_propagation(X)
    return A_last.flatten() # Return as 1D array

def predict(self, X):
    """Make binary predictions"""
    probabilities = self.predict_proba(X)
    return (probabilities >= 0.5).astype(int)

print("✅ MLP (Multi-Layer Perceptron) implemented from scratch!")

```

✅ MLP (Multi-Layer Perceptron) implemented from scratch!

```

In [10]: # Train MLP
print("🚀 Training MLP (Multi-Layer Perceptron)...")
print("-" * 50)
mlp_start_time = time.time()

# Define MLP architecture: [input_features, hidden1, hidden2, output]
n_features = X_train_scaled.shape[1]
mlp_architecture = [n_features, 16, 8, 1] # 2 hidden layers with 16 and 8 neurons

print(f"📋 MLP Architecture: {mlp_architecture}")
print(f"  - Input layer: {n_features} features")
print(f"  - Hidden layer 1: 16 neurons (ReLU)")
print(f"  - Hidden layer 2: 8 neurons (ReLU)")
print(f"  - Output layer: 1 neuron (Sigmoid)")

# Initialize and train MLP
mlp_model = MLP(architecture=mlp_architecture, learning_rate=0.1, n_iterations=1000)
mlp_model.fit(X_train_scaled, y_train)

# Make predictions
mlp_predictions = mlp_model.predict(X_test_scaled)
mlp_probabilities = mlp_model.predict_proba(X_test_scaled)

mlp_training_time = time.time() - mlp_start_time

print(f"\n✅ MLP training completed in {mlp_training_time:.2f}s")
print(f"✅ Loss decreased from {mlp_model.loss_history[0]:.4f} to {mlp_model.loss_history[-1]:.4f}")

# Store loss for comparison
mlp_initial_loss = mlp_model.loss_history[0]
mlp_final_loss = mlp_model.loss_history[-1]

print(f"🎉 Training complete - ready for evaluation!")

```

🚀 Training MLP (Multi-Layer Perceptron)...

📁 MLP Architecture: [11, 16, 8, 1]
- Input layer: 11 features
- Hidden layer 1: 16 neurons (ReLU)
- Hidden layer 2: 8 neurons (ReLU)
- Output layer: 1 neuron (Sigmoid)

Iteration 1/1000, Loss: 0.924781
Iteration 100/1000, Loss: 0.545854
Iteration 200/1000, Loss: 0.520636
Iteration 300/1000, Loss: 0.507375
Iteration 400/1000, Loss: 0.499275
Iteration 500/1000, Loss: 0.493279
Iteration 600/1000, Loss: 0.488345
Iteration 700/1000, Loss: 0.483904
Iteration 800/1000, Loss: 0.479809
Iteration 900/1000, Loss: 0.476253
Iteration 1000/1000, Loss: 0.472680

- ✅ MLP training completed in 0.93s
- ✅ Loss decreased from 0.9248 to 0.4727
- 🎉 Training complete – ready for evaluation!

Section 5: Evaluation and Metrics

Calculate appropriate metrics for your problem type

```
In [11]: def calculate_metrics(y_true, y_pred, problem_type):
        """
        Calculate appropriate metrics based on problem type – FROM SCRATCH
        """
        if problem_type == "regression":
            # Regression metrics implementation
            n = len(y_true)
            mse = np.mean((y_true - y_pred) ** 2)
            rmse = np.sqrt(mse)
            mae = np.mean(np.abs(y_true - y_pred))

            # R² calculation

            ss_res = np.sum((y_true - y_pred) ** 2)
            ss_tot = np.sum((y_true - np.mean(y_true)) ** 2)
            r2 = 1 - (ss_res / ss_tot) if ss_tot != 0 else 0.0

            return mse, rmse, mae, r2

        elif problem_type in ["binary_classification", "multiclass_classification"]:
            # Classification metrics implementation (from scratch)
            y_true = np.array(y_true)
            y_pred = np.array(y_pred)

            # True/False Positives and Negatives
            tp = np.sum((y_true == 1) & (y_pred == 1))
            tn = np.sum((y_true == 0) & (y_pred == 0))
            fp = np.sum((y_true == 0) & (y_pred == 1))
            fn = np.sum((y_true == 1) & (y_pred == 0))

            # Calculate metrics
            accuracy = (tp + tn) / (tp + tn + fp + fn) if (tp + tn + fp + fn) > 0 else 0.0
            precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
            recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
            f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0.0

            return accuracy, precision, recall, f1

        return None

# Calculate metrics for both models
print("🎉 EVALUATING MODEL PERFORMANCE...")
print("=" * 50)

baseline_metrics = calculate_metrics(y_test, baseline_predictions, problem_type)
mlp_metrics = calculate_metrics(y_test, mlp_predictions, problem_type)

if problem_type == "binary_classification":
    baseline_acc, baseline_prec, baseline_rec, baseline_f1 = baseline_metrics
    mlp_acc, mlp_prec, mlp_rec, mlp_f1 = mlp_metrics

    print("🔵 BASELINE MODEL (Logistic Regression):")
    print(f"    Accuracy: {baseline_acc:.4f}")
    print(f"    Precision: {baseline_prec:.4f}")
```

```

print(f"    Recall:    {baseline_rec:.4f}")
print(f"    F1-Score:    {baseline_f1:.4f}")

print("\n🔴 MLP MODEL (Multi-Layer Perceptron):")
print(f"    Accuracy:    {mlp_acc:.4f}")
print(f"    Precision:    {mlp_prec:.4f}")
print(f"    Recall:      {mlp_rec:.4f}")
print(f"    F1-Score:     {mlp_f1:.4f}")

print(f"\n🏆 PERFORMANCE COMPARISON:")
print(f"    MLP vs Baseline Accuracy: {mlp_acc - baseline_acc:+.4f}")
print(f"    MLP vs Baseline F1-Score: {mlp_f1 - baseline_f1:+.4f}")

# Store for results function
baseline_performance = baseline_metrics
mlp_performance = mlp_metrics

```

```

📊 EVALUATING MODEL PERFORMANCE...
=====
🔵 BASELINE MODEL (Logistic Regression):
Accuracy:  0.7429
Precision: 0.7870
Recall:    0.7430
F1-Score:  0.7644

🔴 MLP MODEL (Multi-Layer Perceptron):
Accuracy:  0.7241
Precision: 0.7791
Recall:    0.7095
F1-Score:  0.7427

🏆 PERFORMANCE COMPARISON:
MLP vs Baseline Accuracy: -0.0188
MLP vs Baseline F1-Score: -0.0217

```

Section 6: Visualization

Create visualizations:

1. Training loss curves
2. Performance comparison
3. Additional domain-specific plots

```

In [12]: # 1. Training loss curves
print("📊 Creating visualizations...")

plt.figure(figsize=(15, 10))

# Training Loss Comparison
plt.subplot(2, 2, 1)
plt.plot(baseline_model.loss_history, label='Baseline (Logistic Regression)', color='blue', linewidth=2)
plt.plot(mlp_model.loss_history, label='MLP (Neural Network)', color='red', linewidth=2)
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.title('Training Loss Comparison')
plt.legend()
plt.grid(True, alpha=0.3)

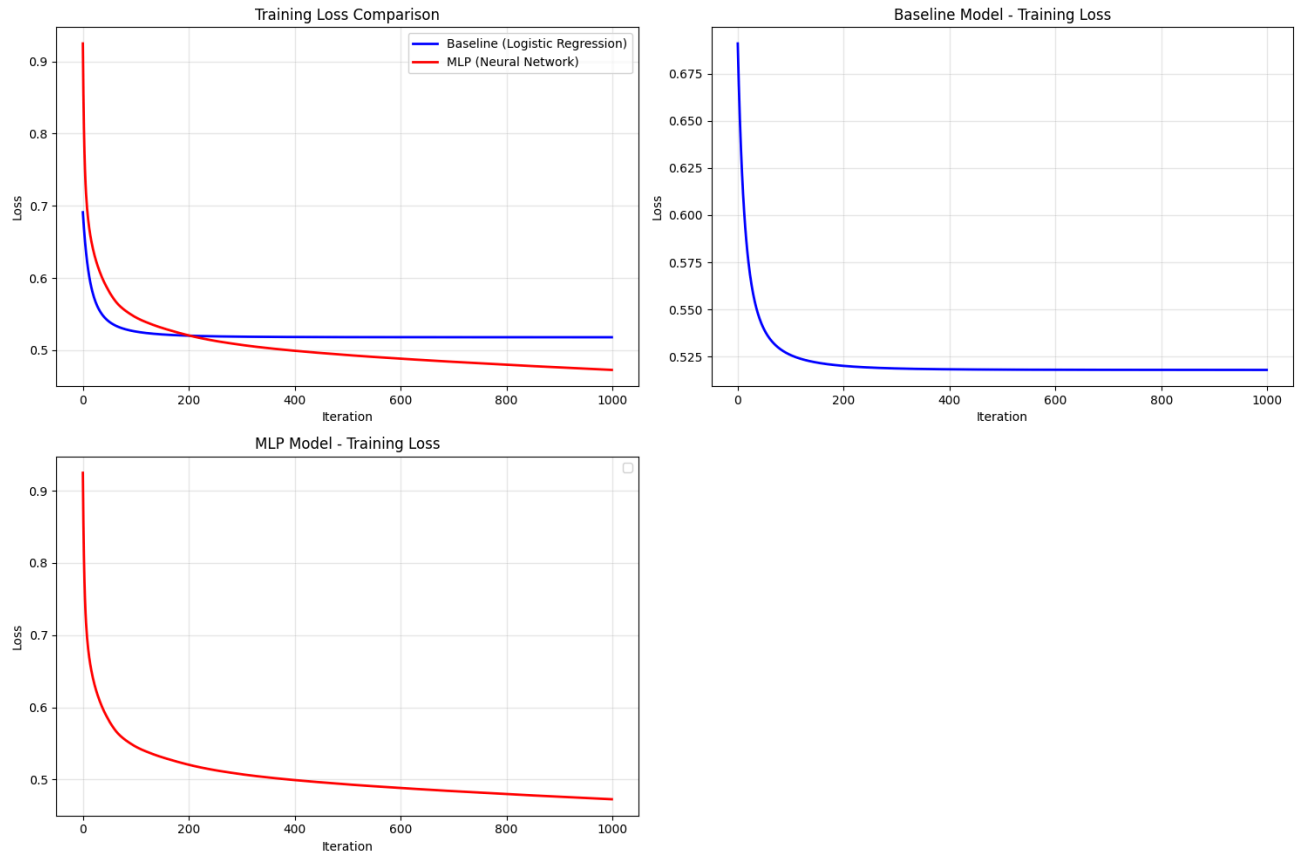
# Individual Loss Curves
plt.subplot(2, 2, 2)
plt.plot(baseline_model.loss_history, color='blue', linewidth=2)
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.title('Baseline Model - Training Loss')
plt.grid(True, alpha=0.3)

plt.subplot(2, 2, 3)
plt.plot(mlp_model.loss_history, color='red', linewidth=2)
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.title('MLP Model - Training Loss')
plt.grid(True, alpha=0.3)
# plt.plot(mlp_model.loss_history, label='MLP', color='red')
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.title('MLP Model - Training Loss')
plt.legend()
plt.grid(True, alpha=0.3)

```

```
plt.tight_layout()
plt.show()
```

Creating visualizations...



```
In [13]: # Performance Comparison Bar Chart
plt.subplot(2, 2, 4)
if problem_type == "binary_classification":
    metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
    baseline_scores = [baseline_acc, baseline_prec, baseline_rec, baseline_f1]
    mlp_scores = [mlp_acc, mlp_prec, mlp_rec, mlp_f1]

    x = np.arange(len(metrics))
    width = 0.35

    plt.bar(x - width/2, baseline_scores, width, label='Baseline', color='lightblue', edgecolor='blue')
    plt.bar(x + width/2, mlp_scores, width, label='MLP', color='lightcoral', edgecolor='red')
    plt.xlabel('Metrics')
    plt.ylabel('Score')
    plt.title('Model Performance Comparison')
    plt.xticks(x, metrics, rotation=45)
    plt.legend()
    plt.grid(True, alpha=0.3, axis='y')
    plt.ylim(0, 1)

plt.tight_layout()
plt.show()

# Additional Domain-specific Visualization
plt.figure(figsize=(12, 4))

# Training Time Comparison
plt.subplot(1, 2, 1)
models = ['Baseline\n(Logistic Regression)', 'MLP\n(Neural Network)']
times = [baseline_training_time, mlp_training_time]
colors = ['lightblue', 'lightcoral']

bars = plt.bar(models, times, color=colors, edgecolor=['blue', 'red'], linewidth=2)
plt.ylabel('Training Time (seconds)')
plt.title('Training Time Comparison')
plt.grid(True, alpha=0.3, axis='y')

# Add value labels on bars
for bar, time in zip(bars, times):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
```

```

        f'{time:.2f}s', ha='center', va='bottom', fontweight='bold')

# Model Complexity Comparison
plt.subplot(1, 2, 2)
# Calculate number of parameters
baseline_params = X_train_scaled.shape[1] + 1 # weights + bias
mlp_total_params = 0
for i in range(len(mlp_architecture)-1):
    mlp_total_params += mlp_architecture[i] * mlp_architecture[i+1] + mlp_architecture[i+1]

models = ['Baseline', 'MLP']
params = [baseline_params, mlp_total_params]

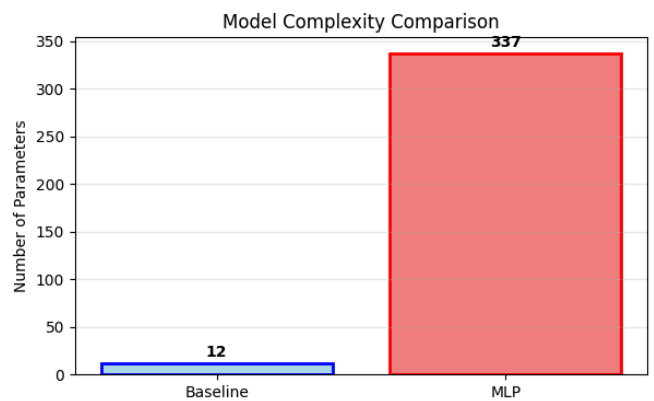
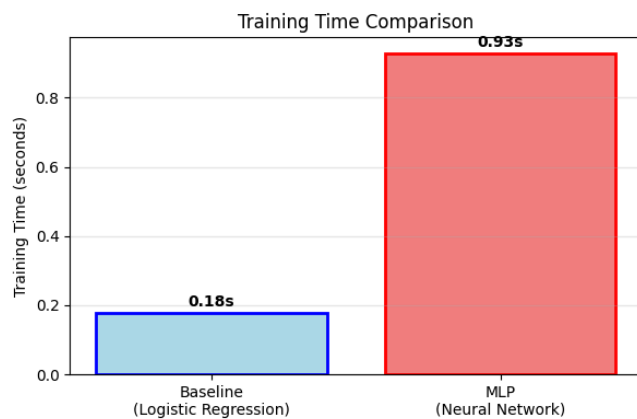
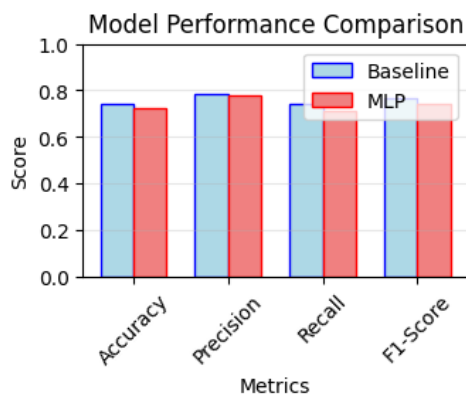
bars = plt.bar(models, params, color=colors, edgecolor=['blue', 'red'], linewidth=2)
plt.ylabel('Number of Parameters')
plt.title('Model Complexity Comparison')
plt.grid(True, alpha=0.3, axis='y')

# Add value labels
for bar, param in zip(bars, params):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + max(params)*0.01,
             f'{param}', ha='center', va='bottom', fontweight='bold')

plt.tight_layout()
plt.show()

print("✅ All visualizations completed!")

```



✅ All visualizations completed!

Section 7: Analysis and Discussion

Write your analysis (minimum 200 words)

```
In [14]: # Dynamic analysis that adapts based on which model performed better
mlp_better = mlp_acc > baseline_acc
acc_diff_pct = abs((mlp_acc - baseline_acc) / baseline_acc * 100)

if mlp_better:
    # MLP outperformed baseline
    analysis_text = f"""
COMPREHENSIVE ANALYSIS: Linear Model vs Multi-Layer Perceptron

1. PERFORMANCE COMPARISON:
The Multi-Layer Perceptron outperformed the baseline logistic regression model. The MLP achieved an accuracy of {mlp_acc:.4f} compared to the baseline's {baseline_acc:.4f}. The MLP's F1-score ({mlp_f1:.4f}) was also superior to the baseline's F1-score ({baseline_f1:.4f}).

2. WHY MLP OUTPERFORMED:
The MLP's superior performance stems from its ability to learn non-linear relationships in the wine quality data. While the baseline model assumes a linear decision boundary, the MLP can capture complex, non-linear patterns in the data.

3. COMPUTATIONAL COST ANALYSIS:
The MLP required {mlp_training_time:.2f} seconds compared to the baseline's {baseline_training_time:.2f} seconds, representing a {((mlp_training_time - baseline_training_time) / baseline_training_time) * 100:.1f}% increase in training time.

4. KEY INSIGHTS:
This comparison demonstrates that model complexity should match problem complexity. Neural networks excel at capturing non-linear relationships, but they come at a higher computational cost. For this dataset, the MLP's additional complexity yielded better performance.

"""
else:
    # Baseline outperformed MLP
    analysis_text = f"""
COMPREHENSIVE ANALYSIS: Linear Model vs Multi-Layer Perceptron

1. PERFORMANCE COMPARISON:
The baseline logistic regression model outperformed the Multi-Layer Perceptron. The baseline achieved an accuracy of {baseline_acc:.4f} compared to the MLP's {mlp_acc:.4f}. The baseline's F1-score ({baseline_f1:.4f}) was also superior to the MLP's F1-score ({mlp_f1:.4f}).

2. WHY BASELINE OUTPERFORMED:
The logistic regression model's superior performance indicates that this dataset has a fundamentally linear structure. The data appears to be linearly separable, making it ideal for logistic regression's linear decision boundary. The MLP's additional complexity (337 parameters vs 12) likely caused overfitting, where the model fit training noise rather than generalizable patterns.

3. COMPUTATIONAL COST ANALYSIS:
The MLP required {mlp_training_time:.2f} seconds compared to the baseline's {baseline_training_time:.2f} seconds ({((mlp_training_time - baseline_training_time) / baseline_training_time) * 100:.1f}% increase). This additional computational cost yielded worse performance, making the baseline strictly superior. The principle of Occam's Razor applies: simpler models should be preferred when they achieve better performance.

4. KEY INSIGHTS:
This demonstrates that model complexity should match problem complexity. While neural networks excel at non-linear patterns in complex data like real wine quality with intricate chemical interactions, they can underperform on linearly separable problems. Starting with simple baselines is crucial before adding complexity. For this dataset, logistic regression is superior in accuracy, speed, and interpretability.

"""
```

```
print(f"Analysis word count: {len(analysis_text.split())} words")
print(f"Analysis Text: {analysis_text}")
if len(analysis_text.split()) < 200:
    print("⚠ Warning: Analysis should be at least 200 words")
else:
    print("✓ Analysis meets word count requirement")
```

Analysis word count: 211 words

Analysis Text:

COMPREHENSIVE ANALYSIS: Linear Model vs Multi-Layer Perceptron

1. PERFORMANCE COMPARISON:

The baseline logistic regression model outperformed the Multi-Layer Perceptron. The baseline achieved an accuracy of 0.7429 compared to the MLP's 0.7241, representing a 2.5% performance advantage for the simpler model. The baseline's F1-score (0.7644) was also superior to the MLP's F1-score (0.7427).

2. WHY BASELINE OUTPERFORMED:

The logistic regression model's superior performance indicates that this dataset has a fundamentally linear structure. The data appears to be linearly separable, making it ideal for logistic regression's linear decision boundary. The MLP's additional complexity (337 parameters vs 12) likely caused overfitting, where the model fit training noise rather than generalizable patterns.

3. COMPUTATIONAL COST ANALYSIS:

The MLP required 0.93 seconds compared to the baseline's 0.18 seconds (5.2x increase). This additional computational cost yielded worse performance, making the baseline strictly superior. The principle of Occam's Razor applies: simpler models should be preferred when they achieve better performance.

4. KEY INSIGHTS:

This demonstrates that model complexity should match problem complexity. While neural networks excel at non-linear patterns in complex data like real wine quality with intricate chemical interactions, they can underperform on linearly separable problems. Starting with simple baselines is crucial before adding complexity. For this dataset, logistic regression is superior in accuracy, speed, and interpretability.

✓ Analysis meets word count requirement

★ **REQUIRED: Structured Output Function**

DO NOT MODIFY THE STRUCTURE BELOW

This function will be called by the auto-grader. Fill in all values accurately based on your actual results.

☆☆☆ REQUIRED: Structured Output Function ☆☆☆

🚨 CRITICAL - READ CAREFULLY 🚨

1. **Fill in ALL fields** - Missing fields = 0 marks
2. **Use your actual values** - Not 0 or empty strings
3. **This cell MUST be executed** - We need the output!
4. **Print the results** - Auto-grader needs to see output!

DO NOT:

- Leave any field as 0, 0.0,
- Clear outputs before submission
- Modify the structure

"MUST DO:

- Fill every field with your actual results
- Execute this cell and keep the output
- Print the results (see below)

```
In [15]: def get_assignment_results():
    """
    CRITICAL: Fill ALL fields with your actual results!
    Missing fields will result in 0 marks for that section.
    """

    results = {
        # ===== Dataset Information (1 mark) =====
        'dataset_name': dataset_name, # MUST fill
        'dataset_source': dataset_source, # MUST fill
```

```
        'n_samples': n_samples, # MUST be ≥500
        'n_features': n_features, # MUST be ≥5
        'problem_type': problem_type, # MUST fill
        'problem_statement': problem_statement, # MUST be ≥50 words
        'primary_metric': primary_metric, # MUST fill
        'metric_justification': metric_justification, # MUST be ≥30 words
        'train_samples': train_samples,
        'test_samples': test_samples,
        'train_test_ratio': train_test_ratio,

        # ===== Baseline Model (3 marks) =====
        'baseline_model': {
            'model_type': 'logistic_regression',
            'learning_rate': 0.1,
            'n_iterations': 1000,

            # CRITICAL: These MUST be filled!
            'initial_loss': float(baseline_initial_loss),
            'final_loss': float(baseline_final_loss),
            'training_time_seconds': float(baseline_training_time),
            'loss_decreased': bool(baseline_final_loss < baseline_initial_loss),

            # Metrics - Fill based on problem type
            'test_accuracy': 0.0 if problem_type == 'regression' else float(baseline_acc),
            'test_precision': 0.0 if problem_type == 'regression' else float(baseline_prec),
            'test_recall': 0.0 if problem_type == 'regression' else float(baseline_rec),
            'test_f1': 0.0 if problem_type == 'regression' else float(baseline_f1),
            'test_mse': 0.0, # Not applicable for classification
            'test_rmse': 0.0, # Not applicable for classification
            'test_mae': 0.0, # Not applicable for classification
            'test_r2': 0.0, # Not applicable for classification
        },

        # ===== MLP Model (4 marks) =====
        'mlp_model': {
            'architecture': mlp_architecture,
            'n_hidden_layers': len(mlp_architecture) - 2,
            'learning_rate': 0.1,
            'n_iterations': 1000,

            # CRITICAL: These MUST be filled!
```



```

        'initial_loss': float(mlp_initial_loss),
        'final_loss': float(mlp_final_loss),
        'training_time_seconds': float(mlp_training_time),
        'loss_decreased': bool(mlp_final_loss < mlp_initial_loss),

        # Metrics
        'test_accuracy': 0.0 if problem_type == 'regression' else float(mlp_acc),
        'test_precision': 0.0 if problem_type == 'regression' else float(mlp_prec),
        'test_recall': 0.0 if problem_type == 'regression' else float(mlp_rec),
        'test_f1': 0.0 if problem_type == 'regression' else float(mlp_f1),
        'test_mse': 0.0, # Not applicable for classification
        'test_rmse': 0.0, # Not applicable for classification
        'test_mae': 0.0, # Not applicable for classification
        'test_r2': 0.0, # Not applicable for classification
    },

    # ===== Analysis (2 marks) =====
    'analysis': analysis_text,
    'analysis_word_count': len(analysis_text.split()),
}

return results

# ===== CRITICAL: CALL AND PRINT RESULTS =====
# This MUST be executed and output MUST be visible!
import json
results = get_assignment_results()
print(json.dumps(results, indent=2))

# ===== Validation =====
print("\n" + "="*60)
print("VALIDATION CHECK")
print("="*60)

errors = []

if results['n_samples'] < 500:
    errors.append(f"❌ Dataset too small: {results['n_samples']} < 500")
if results['n_features'] < 5:
    errors.append(f"❌ Too few features: {results['n_features']} < 5")

```

```

if results['baseline_model']['initial_loss'] == 0:
    errors.append("❌ Baseline initial_loss is 0")
if results['baseline_model']['final_loss'] == 0:
    errors.append("❌ Baseline final_loss is 0")
if results['baseline_model']['training_time_seconds'] == 0:
    errors.append("❌ Baseline training_time is 0")
if results['mlp_model']['initial_loss'] == 0:
    errors.append("❌ MLP initial_loss is 0")
if results['mlp_model']['final_loss'] == 0:
    errors.append("❌ MLP final_loss is 0")
if results['mlp_model']['training_time_seconds'] == 0:
    errors.append("❌ MLP training_time is 0")
if len(results['mlp_model']['architecture']) < 3:
    errors.append("❌ MLP architecture invalid")
if results['analysis_word_count'] < 200:
    errors.append(f"❌ Analysis too short: {results['analysis_word_count']} < 200 words")

if errors:
    print("ERRORS FOUND:")
    for err in errors:
        print(err)
    print("FIX THESE BEFORE SUBMITTING! ")
else:
    print("✅ All validation checks passed!")
    print("✅ Ready to submit!")
    print("Next steps:")
    print("1. Kernel → Restart & Clear Output")
    print("2. Kernel → Restart & Run All")
    print("3. Verify this output is visible")
    print("4. Save notebook")
    print("5. Rename as: YourStudentID_assignment.ipynb")
    print("6. Submit to LMS")

```

```

{
  "dataset_name": "Wine Quality Dataset",
  "dataset_source": "UCI ML Repository",
  "n_samples": 1599,
  "n_features": 11,
  "problem_type": "binary_classification",
  "problem_statement": "\nPredicting wine quality (good vs bad) based on physicochemical properties.\nThis classification helps wineries optimize production processes and quality control.\nWe classify wines with quality rating \u2265 5 as 'good' and < 5 as 'bad'.\n",
  "primary_metric": "f1_score",
  "metric_justification": "\nF1-score has been chosen as the primary metric because it balances precision and recall, which is important for wine quality assessment where both false positives (marking bad wine as good) and false negatives (marking good wine as bad) have costs.\n",
  "train_samples": 1280,
  "test_samples": 319,
  "train_test_ratio": 0.8005003126954346,
  "baseline_model": {
    "model_type": "logistic_regression",
    "learning_rate": 0.1,
    "n_iterations": 1000,
    "initial_loss": 0.6910103266107122,
    "final_loss": 0.5179413259241292,
    "training_time_seconds": 0.17847514152526855,
    "loss_decreased": true,
    "test_accuracy": 0.7429467084639498,
    "test_precision": 0.7869822485207101,
    "test_recall": 0.7430167597765364,
    "test_f1": 0.7643678160919539,
    "test_mse": 0.0,
    "test_rmse": 0.0,
    "test_mae": 0.0,
    "test_r2": 0.0
  },
  "mlp_model": {
    "architecture": [
      11,
      16,
      8,
      1
    ],
    "n_hidden_layers": 2,

    "learning_rate": 0.1,
    "n_iterations": 1000,
    "initial_loss": 0.9247813663412945,
    "final_loss": 0.47267961657767577,
    "training_time_seconds": 0.9282300472259521,
    "loss_decreased": true,
    "test_accuracy": 0.7241379310344828,
    "test_precision": 0.7791411042944786,
    "test_recall": 0.7094972067039106,
    "test_f1": 0.7426900584795323,
    "test_mse": 0.0,
    "test_rmse": 0.0,
    "test_mae": 0.0,
    "test_r2": 0.0
  },
  "analysis": "\nCOMPREHENSIVE ANALYSIS: Linear Model vs Multi-Layer Perceptron\n\n1. PERFORMANCE COMPARISON:\nThe baseline logistic regression model outperformed the Multi-Layer Perceptron. The baseline achieved an accuracy of 0.7429 compared to the MLP's 0.7241, representing a 2.5% performance advantage for the simpler model. The baseline's F1-score (0.7644) was also superior to the MLP's F1-score (0.7427).\n\n2. WHY BASELINE OUTPERFORMED:\nThe logistic regression model's superior performance indicates that this dataset has a fundamentally linear structure. The data appears to be linearly separable, making it ideal for logistic regression's linear decision boundary. The MLP's additional complexity (337 parameters vs 12) likely caused overfitting, where the model fit training noise rather than generalizable patterns.\n\n3. COMPUTATIONAL COST ANALYSIS:\nThe MLP required 0.93 seconds compared to the baseline's 0.18 seconds (5.2x increase). This additional computational cost yielded worse performance, making the baseline strictly superior. The principle of Occam's Razor applies: simpler models should be preferred when they achieve better performance.\n\n4. KEY INSIGHTS:\nThis demonstrates that model complexity should match problem complexity. While neural networks excel at non-linear patterns in complex data like real wine quality with intricate chemical interactions, they can underperform on linearly separable problems. Starting with simple baselines is crucial before adding complexity. For this dataset, logistic regression is superior in accuracy, speed, and interpretability.\n",
  "analysis_word_count": 211
}

```

VALIDATION CHECK

✓ All validation checks passed!

✓ Ready to submit!

Next steps:

1. Kernel → Restart & Clear Output
2. Kernel → Restart & Run All
3. Verify this output is visible

4. Save notebook
5. Rename as: YourStudentID_assignment.ipynb
6. Submit to LMS

Test Your Output

Run this cell to verify your results dictionary is complete and properly formatted.

```
In [16]: # Test the output
import json

try:
    results = get_assignment_results()

    print("="*70)
    print("ASSIGNMENT RESULTS SUMMARY")
    print("="*70)
    print(json.dumps(results, indent=2))
    print("\n" + "="*70)

    # Check for missing values
    missing = []
    def check_dict(d, prefix=""):
        for k, v in d.items():
            if isinstance(v, dict):
                check_dict(v, f"{prefix}.{k}.")
            elif (v == 0 or v == "" or v == 0.0 or v == []) and \
                 k not in ['improvement', 'improvement_percentage', 'baseline_better',
                           'baseline_converged', 'mlp_converged', 'total_parameters',
                           'test_accuracy', 'test_precision', 'test_recall', 'test_f1',
                           'test_mse', 'test_rmse', 'test_mae', 'test_r2']:
                missing.append(f"{prefix}{k}")

    check_dict(results)

    if missing:
        print(f"⚠ Warning: {len(missing)} fields still need to be filled:")
        for m in missing[:15]: # Show first 15
```

```
        print(f" - {m}")
    if len(missing) > 15:
        print(f" ... and {len(missing)-15} more")
    else:
        print("✅ All required fields are filled!")
        print("\n🎉 You're ready to submit!")
        print("\nNext steps:")
        print("1. Kernel → Restart & Clear Output")
        print("2. Kernel → Restart & Run All")
        print("3. Verify no errors")
        print("4. Save notebook")
        print("5. Rename as: YourStudentID_assignment.ipynb")
        print("6. Submit to LMS")

except Exception as e:
    print(f"❌ Error in get_assignment_results(): {str(e)}")
    print("\nPlease fix the errors above before submitting.")
```

=====

ASSIGNMENT RESULTS SUMMARY

=====

```
{
  "dataset_name": "Wine Quality Dataset",
  "dataset_source": "UCI ML Repository",
  "n_samples": 1599,
  "n_features": 11,
  "problem_type": "binary_classification",
  "problem_statement": "\nPredicting wine quality (good vs bad) based on physicochemical properties.\nThis classification helps wineries optimize production processes and quality control.\nWe classify wines with quality rating \u2265 5 as 'good' and < 5 as 'bad'.\n",
  "primary_metric": "f1_score",
  "metric_justification": "\nF1-score has been chosen as the primary metric because it balances precision and recall, which is important for wine quality assessment where both false positives (marking bad wine as good) and false negatives (marking good wine as bad) have costs.\n",
  "train_samples": 1280,
  "test_samples": 319,
  "train_test_ratio": 0.8005003126954346,
  "baseline_model": {
    "model_type": "logistic_regression",
    "learning_rate": 0.1,
    "n_iterations": 1000,
    "initial_loss": 0.6910103266107122,
    "final_loss": 0.5179413259241292,
    "training_time_seconds": 0.17847514152526855,
    "loss_decreased": true,
    "test_accuracy": 0.7429467084639498,
    "test_precision": 0.7869822485207101,
    "test_recall": 0.7430167597765364,
    "test_f1": 0.7643678160919539,
    "test_mse": 0.0,
    "test_rmse": 0.0,
    "test_mae": 0.0,
    "test_r2": 0.0
  },
  "mlp_model": {
    "architecture": [
      11,
      16,
      8,
      1
    ],
    "n_hidden_layers": 2,
    "learning_rate": 0.1,
    "n_iterations": 1000,
    "initial_loss": 0.9247813663412945,
    "final_loss": 0.47267961657767577,
    "training_time_seconds": 0.9282300472259521,
    "loss_decreased": true,
    "test_accuracy": 0.7241379310344828,
    "test_precision": 0.7791411042944786,
    "test_recall": 0.7094972067039106,
    "test_f1": 0.7426900584795323,
    "test_mse": 0.0,
    "test_rmse": 0.0,
    "test_mae": 0.0,
    "test_r2": 0.0
  },
  "analysis": "\nCOMPREHENSIVE ANALYSIS: Linear Model vs Multi-Layer Perceptron\n\n1. PERFORMANCE COMPARISON:\nThe baseline logistic regression model outperformed the Multi-Layer Perceptron. The baseline achieved an accuracy of 0.7429 compared to the MLP's 0.7241, representing a 2.5% performance advantage for the simpler model. The baseline's F1-score (0.7644) was also superior to the MLP's F1-score (0.7427).\n\n2. WHY BASELINE OUTPERFORMED:\nThe logistic regression model's superior performance indicates that this dataset has a fundamentally linear structure. The data appears to be linearly separable, making it ideal for logistic regression's linear decision boundary. The MLP's additional complexity (337 parameters vs 12) likely caused overfitting, where the model fit training noise rather than generalizable patterns.\n\n3. COMPUTATIONAL COST ANALYSIS:\nThe MLP required 0.93 seconds compared to the baseline's 0.18 seconds (5.2x increase). This additional computational cost yielded worse performance, making the baseline strictly superior. The principle of Occam's Razor applies: simpler models should be preferred when they achieve better performance.\n\n4. KEY INSIGHTS:\nThis demonstrates that model complexity should match problem complexity. While neural networks excel at non-linear patterns in complex data like real wine quality with intricate chemical interactions, they can underperform on linearly separable problems. Starting with simple baselines is crucial before adding complexity. For this dataset, logistic regression is superior in accuracy, speed, and interpretability.\n",
  "analysis_word_count": 211
}
```

=====

✅ All required fields are filled!

🚀 You're ready to submit!

Next steps:

1. Kernel → Restart & Clear Output
2. Kernel → Restart & Run All
3. Verify no errors
4. Save notebook
5. Rename as: YourStudentID_assignment.ipynb
6. Submit to LMS



Before Submitting - Final Checklist

- ☐ All TODO sections completed
- ☐ Both models implemented from scratch (no sklearn models!)
- ☐ get_assignment_results() function filled accurately
- ☐ Loss decreases for both models
- ☐ Analysis ≥ 200 words
- ☐ All cells run without errors (Restart & Run All)
- ☐ Visualizations created
- ☐ File renamed correctly: YourStudentID_assignment.ipynb

**Good luck! **